



Secure Hash Algorithm (SHA3-256)

Prepared by

- Abdulmajeed Alotaibi
- Meshal Hussain Alossaimi
- Tariq Abdullah Alawni

Course Information

Course: Network Security (COS 471)

Supervised by: Prof. Wail Alkilani

1. Overview

SHA3-256 is a cryptographic hash function that converts an input message of any length into a fixed output digest of 256 bits. It uses the Sponge Construction and the Keccak-f[1600] permutation.

Plaintext used in this tutorial

AbdulmajeedMeshalTariq

Algorithm Flow / Life Cycle



Core SHA3-256 Parameters

State Size
1600 bits = 200 bytes

State Shape
5×5 lanes

Lane Size
64 bits = 8 bytes

Digest Size
256 bits = 32 bytes

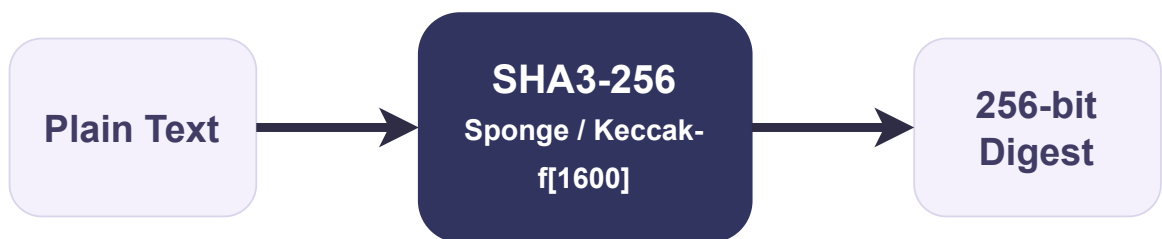
Inside each Keccak-f[1600]
Cycle

θ Theta $\rightarrow$ ρ Rho $\rightarrow$ π Pi $\rightarrow$ χ Chi $\rightarrow$ ι Iota

2. Structure and Sponge

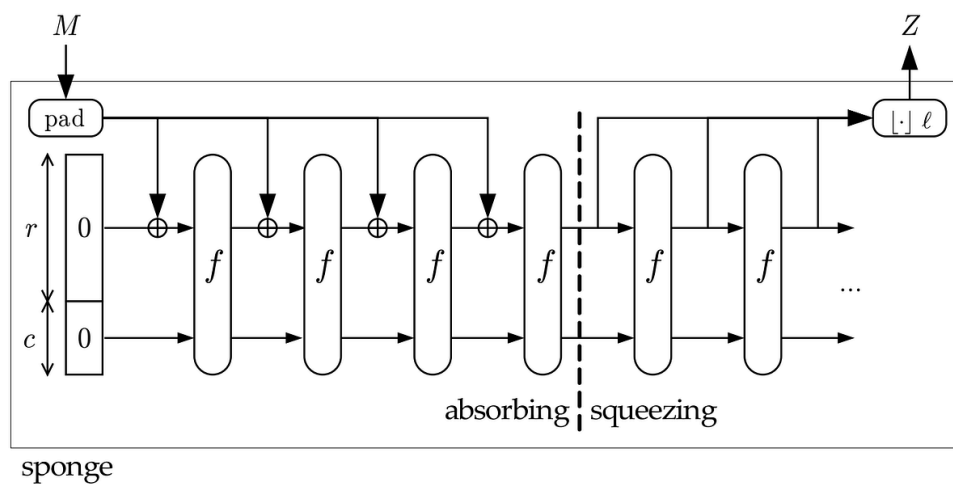
This page introduces the general SHA3-256 structure before the detailed calculations. It presents the high-level transformation from plaintext to digest and the Sponge Construction used in SHA3-256.

High-Level Hashing Flow



General idea: the input message enters the SHA3-256 hashing process and is transformed into a fixed-length output of 256 bits.

Sponge Construction Diagram



This figure will visually connect the message padding stage with the absorbing and squeezing behavior of the SHA3-256 sponge model.

3. Message Padding

In SHA3-256, the message is padded until it becomes exactly one block of 136 bytes = 1088 bits.

Original Plaintext

AbdulmajeedMeshalTariq

22 characters

22 bytes = 176 bits

1. ASCII / Hex Representation

Character	Hex	Binary (8 bits)	Character	Hex	Binary (8 bits)
A	0x41	01000001	M	0x4d	01001101
b	0x62	01100010	e	0x65	01100101
d	0x64	01100100	s	0x73	01110011
u	0x75	01110101	h	0x68	01101000
l	0x6c	01101100	a	0x61	01100001
m	0x6d	01101101	l	0x6c	01101100
a	0x61	01100001	T	0x54	01010100
j	0x6a	01101010	a	0x61	01100001
e	0x65	01100101	r	0x72	01110010
e	0x65	01100101	i	0x69	01101001
d	0x64	01100100	q	0x71	01110001

2. Padding Rule

SHA3-256 Rate
136 bytes
1088 bits

Padding Method
pad10*1

Domain
Separator
0x06
1 byte = 8 bits

Final Byte
0x80
1 byte = 8 bits

We append 0x06, then enough zero bytes, and finally the last byte becomes 0x80.

3. Length Calculation

Original Length
22 bytes
176 bits

+

0x06
1 byte
8 bits

+

Zero Bytes
112 bytes
896 bits

+

0x80
1 byte
8 bits

Total = 136 bytes = 1088 bits

3. Message Padding: Padded Block Layout

The message is now padded into one complete SHA3-256 rate block: 136 bytes = 1088 bits.

Message
22 bytes = 176 bits

+

0x06
1 byte = 8 bits

+

Zero Bytes
112 bytes = 896 bits

+

0x80
1 byte = 8 bits

4. Padded Message in Hex Rows

16 bytes per row. Red cells show the first 8 bytes selected for the first absorption word.

41	62	64	75	6c	6d	61	6a	65	65	64	4d	65	73	68	61
6c	54	61	72	69	71	06	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	80								

First selected word

Domain separator 0x06

Final byte 0x80

Total = 136 bytes = 1088 bits

5. Preparing the First Absorption Word

SHA3-256 reads each 8-byte word as a little-endian 64-bit value, so the byte order is reversed before placing it into the state lane.

Original first 8 bytes

41 62 64 75 6c 6d 61 6a

Reverse byte order for little-endian 64-bit lane

Absorption word after reversal

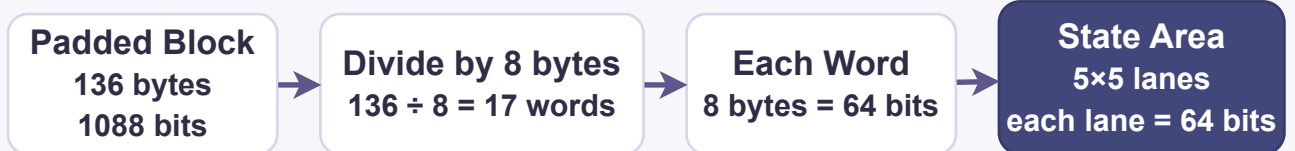
6a 61 6d 6c 75 64 62 41

First 64-bit absorption lane value:
0x6a616d6c75646241

Next step: split the full 136-byte block into 17 words. Each word is 8 bytes = 64 bits, then XOR them into the SHA3 state.

4. Absorption Phase

The padded block is split into 17 words. Each word is 8 bytes = 64 bits, then XORed into the SHA3-256 state lanes.



Little-endian note: each 8-byte group is reversed before becoming a 64-bit lane value.

Word-to-Lane Mapping

17 words fill the rate portion of the state.
Every cell below is editable separately.

Word Index	Lane (x,y)	64-bit Value after Little-Endian Conversion
0	(0,0)	0x6a616d6c75646241
1	(1,0)	0x616873654d646565
2	(2,0)	0x000671697261546c
3	(3,0)	0x0000000000000000
4	(4,0)	0x0000000000000000
5	(0,1)	0x0000000000000000
6	(1,1)	0x0000000000000000
7	(2,1)	0x0000000000000000
8	(3,1)	0x0000000000000000
9	(4,1)	0x0000000000000000
10	(0,2)	0x0000000000000000
11	(1,2)	0x0000000000000000
12	(2,2)	0x0000000000000000
13	(3,2)	0x0000000000000000
14	(4,2)	0x0000000000000000
15	(0,3)	0x0000000000000000
16	(1,3)	0x8000000000000000

Non-zero message words

Final marker word

Zero padding words

Absorption equation: $\text{State}[x,y] = \text{State}[x,y] \text{ XOR } \text{Word}[i]$. Since the initial state is 0, the inserted lanes equal the mapped word values.

4. Absorption Phase: XOR into State

After mapping the 17 words to lanes, each word is XORed with the corresponding 64-bit state lane.

1. XOR Absorption Rule

$$\text{State}[x,y] = \text{State}[x,y] \text{ XOR } \text{Word}[i]$$

Initial SHA3 state = all zeros. Therefore, 0 XOR Word = Word.

2. XOR Examples

Lane	Initial State	Word[i]	XOR Result	Inserted in State
(0,0)	0x0000000000000000	0x6a616d6c75646241	0x6a616d6c75646241	Yes
(1,0)	0x0000000000000000	0x616873654d646565	0x616873654d646565	Yes
(2,0)	0x0000000000000000	0x000671697261546c	0x000671697261546c	Yes
(1,3)	0x0000000000000000	0x8000000000000000	0x8000000000000000	Yes

3. State After XORing All 17 Words

All remaining lanes are zero.
Values below are 64-bit hexadecimal lanes.

	x=0	x=1	x=2	x=3	x=4
y=0	6a616d6c 75646241	61687365 4d646565	00067169 7261546c	00000000 00000000	00000000 00000000
y=1	00000000 00000000	00000000 00000000	00000000 00000000	00000000 00000000	00000000 00000000
y=2	00000000 00000000	00000000 00000000	00000000 00000000	00000000 00000000	00000000 00000000
y=3	00000000 00000000	80000000 00000000	00000000 00000000	00000000 00000000	00000000 00000000
y=4	00000000 00000000	00000000 00000000	00000000 00000000	00000000 00000000	00000000 00000000

Message-derived lanes

Final 0x80 marker lane

Zero lanes

The state is now ready. In the next page, we introduce the Round Constants (RC), which will be used during the Iota step of each Keccak-f[1600] cycle.

Next Page: Round Constants (RC)

5. Round Constants (RC)

Round Constants are fixed 64-bit values used during the Iota step. Each Keccak-f[1600] cycle uses one different RC value.

Number of Cycles
24 cycles

RC Size
64 bits = 8 bytes

Target Lane
A[0,0] only

Iota Step Formula

$$A[0,0] = A[0,0] \text{ XOR } RC[i]$$

Purpose: RC prevents symmetry between rounds. Without Iota, repeated rounds could preserve structural patterns in the state.

Round Constants Table

Each card is editable separately.
One 64-bit constant is used per cycle.

RC[0] 0x0000000000000001	RC[1] 0x0000000000000802	RC[2] 0x800000000000080a	RC[3] 0x8000000008000800
RC[4] 0x000000000000080b	RC[5] 0x0000000008000001	RC[6] 0x8000000008000801	RC[7] 0x8000000000000809
RC[8] 0x000000000000008a	RC[9] 0x0000000000000088	RC[10] 0x0000000008000809	RC[11] 0x000000000800000a
RC[12] 0x000000000800080b	RC[13] 0x800000000000008b	RC[14] 0x8000000000000809	RC[15] 0x8000000000000803
RC[16] 0x8000000000000802	RC[17] 0x8000000000000080	RC[18] 0x000000000000080a	RC[19] 0x800000000800000a
RC[20] 0x8000000008000801	RC[21] 0x8000000000000800	RC[22] 0x0000000008000001	RC[23] 0x8000000008000808

In Cycle 0, RC[0] is used. In Cycle 1, RC[1] is used. The same pattern continues until Cycle 23.

Next Page: Rotation Offsets (ROT)

6. Rotation Offsets (ROT)

Rotation offsets are fixed values used in the Rho step. Each 64-bit lane is rotated left by the number of bits shown in the table.

Used in Step
ρ Rho

Lane Size
64 bits = 8 bytes

Unit
bits, not bytes

Rho Rotation Formula

$$A[x,y] = \text{ROTL64}(A[x,y], \text{ROT}[x,y])$$

ROTL64 means rotate-left inside a 64-bit lane. Bits that leave the left side wrap around to the right side.

Rotation Offsets Table

Each number is a rotation amount in bits.

y \ x	x=0	x=1	x=2	x=3	x=4
y=0	0	1	62	28	27
y=1	36	44	6	55	20
y=2	3	10	43	25	39
y=3	41	45	15	21	8
y=4	18	2	61	56	14

Example

Lane (1,0) uses $\text{ROT}[1,0] = 1$ bit. So its 64-bit value is rotated left by 1 bit during Rho.

These offsets never change. They are fixed by the Keccak-f[1600] design and reused in every one of the 24 cycles.

Next Page: Cycle 0 Begins — State Before Theta

Cycle 0

This cycle has five steps: Theta → Rho → Pi → Chi → Iota. All lanes are 64 bits = 8 bytes.

What this section does

We start from the absorbed 5×5 state. Cycle 0 transforms it through five operations. The next pages expand every calculation instead of jumping directly to the result.

θ Theta

Column XOR mixing: compute C[x], then D[x], then XOR D[x] into each column.

ρ Rho

Rotate each 64-bit lane left by a fixed number of bits.

π Pi

Move each rotated lane to a new coordinate.

χ Chi

Non-linear row mixing using NOT, AND, XOR.

ι Iota

XOR RC[0] into lane A[0,0].

Initial State Before Cycle 0

	x=0	x=1	x=2	x=3	x=4
y=0	6a616d6c 75646241	61687365 4d646565	00067169 7261546c	00000000 00000000	00000000 00000000
y=1	00000000 00000000	00000000 00000000	00000000 00000000	00000000 00000000	00000000 00000000
y=2	00000000 00000000	00000000 00000000	00000000 00000000	00000000 00000000	00000000 00000000
y=3	00000000 00000000	80000000 00000000	00000000 00000000	00000000 00000000	00000000 00000000
y=4	00000000 00000000	00000000 00000000	00000000 00000000	00000000 00000000	00000000 00000000

Note: red/orange cells come from the message bytes; purple cell comes from the final 0x80 padding marker.

Cycle 0 — Theta C[0] Calculation

Theta begins by XORing the five lanes in each column.

C[0] formula

$$C[0] = A[0,0] \text{ XOR } A[0,1] \text{ XOR } A[0,2] \text{ XOR } A[0,3] \text{ XOR } A[0,4]$$

Lane	64-bit hex value
A[0,0]	0x6a616d6c75646241
A[0,1]	0x0000000000000000
A[0,2]	0x0000000000000000
A[0,3]	0x0000000000000000
A[0,4]	0x0000000000000000
XOR Result	0x6a616d6c75646241

Binary XOR calculation for C[0]

```

A[0,0] 01101010 01100001 01101101 01101100 01110101 01100100 01100010 01000001
A[0,1/2/3/4] zeros 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
C[0] result 01101010 01100001 01101101 01101100 01110101 01100100 01100010 01000001
  
```

How C[0] becomes hexadecimal

Each hexadecimal digit represents exactly 4 bits.

6	a	6	1	6	d	6	c	7	5	6	4	6	2	4	1
0110	1010	0110	0001	0110	1101	0110	1100	0111	0101	0110	0100	0110	0010	0100	0001

Key idea: XOR compares bits column by column. Equal bits give 0; different bits give 1.

Cycle 0 — Theta C[1] Calculation

Theta begins by XORing the five lanes in each column.

C[1] formula

$$C[1] = A[1,0] \text{ XOR } A[1,1] \text{ XOR } A[1,2] \text{ XOR } A[1,3] \text{ XOR } A[1,4]$$

Lane	64-bit hex value
A[1,0]	0x616873654d646565
A[1,1]	0x0000000000000000
A[1,2]	0x0000000000000000
A[1,3]	0x8000000000000000
A[1,4]	0x0000000000000000
XOR Result	0xe16873654d646565

Binary XOR calculation for C[1]

```

A[1,0] 01100001 01101000 01110011 01100101 01001101 01100100 01100101 01100101
A[1,3] 10000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
A[1,1/2/4] zeros 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
C[1] result 11100001 01101000 01110011 01100101 01001101 01100100 01100101 01100101
  
```

How C[1] becomes hexadecimal

Each hexadecimal digit represents exactly 4 bits.

e	1	6	8	7	3	6	5	4	d	6	4	6	5	6	5
1110	0001	0110	1000	0111	0011	0110	0101	0100	1101	0110	0100	0110	0101	0110	0101

Key idea: XOR compares bits column by column. Equal bits give 0; different bits give 1.

Cycle 0 — Theta C[2] Calculation

Theta begins by XORing the five lanes in each column.

C[2] formula

$C[2] = A[2,0] \text{ XOR } A[2,1] \text{ XOR } A[2,2] \text{ XOR } A[2,3] \text{ XOR } A[2,4]$

Lane	64-bit hex value
A[2,0]	0x000671697261546c
A[2,1]	0x0000000000000000
A[2,2]	0x0000000000000000
A[2,3]	0x0000000000000000
A[2,4]	0x0000000000000000
XOR Result	0x000671697261546c

Binary XOR calculation for C[2]

```

A[2,0] 00000000 00000110 01110001 01101001 01110010 01100001 01010100 01101100
A[2,1/2/3/4] zeros 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
C[2] result 00000000 00000110 01110001 01101001 01110010 01100001 01010100 01101100
  
```

How C[2] becomes hexadecimal

Each hexadecimal digit represents exactly 4 bits.

0	0	0	6	7	1	6	9	7	2	6	1	5	4	6	c
0000	0000	0000	0110	0111	0001	0110	1001	0111	0010	0110	0001	0101	0100	0110	1100

Key idea: XOR compares bits column by column. Equal bits give 0; different bits give 1.

Cycle 0 — Theta C[3] Calculation

Theta begins by XORing the five lanes in each column.

C[3] formula

$$C[3] = A[3,0] \text{ XOR } A[3,1] \text{ XOR } A[3,2] \text{ XOR } A[3,3] \text{ XOR } A[3,4]$$

Lane	64-bit hex value
A[3,0]	0x0000000000000000
A[3,1]	0x0000000000000000
A[3,2]	0x0000000000000000
A[3,3]	0x0000000000000000
A[3,4]	0x0000000000000000
XOR Result	0x0000000000000000

Binary XOR calculation:

A[3,0..4] zeros 00000000 00000000 00000000 00000000 00000000 00000000
00000000 00000000

C[3] result 00000000 00000000 00000000 00000000 00000000 00000000
00000000 00000000

Zero result represented as 16 hex digits

Each hexadecimal digit represents exactly 4 bits.

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000

Key idea: XOR compares bits column by column. Equal bits give 0; different bits give 1.

Cycle 0 — Theta C[4] Calculation

Theta begins by XORing the five lanes in each column.

C[4] formula

$$C[4] = A[4,0] \text{ XOR } A[4,1] \text{ XOR } A[4,2] \text{ XOR } A[4,3] \text{ XOR } A[4,4]$$

Lane	64-bit hex value
A[4,0]	0x0000000000000000
A[4,1]	0x0000000000000000
A[4,2]	0x0000000000000000
A[4,3]	0x0000000000000000
A[4,4]	0x0000000000000000
XOR Result	0x0000000000000000

Binary XOR calculation:

A[4,0..4] zeros 00000000 00000000 00000000 00000000 00000000 00000000
00000000 00000000

C[4] result 00000000 00000000 00000000 00000000 00000000 00000000
00000000 00000000

Zero result represented as 16 hex digits

Each hexadecimal digit represents exactly 4 bits.

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000

Key idea: XOR compares bits column by column. Equal bits give 0; different bits give 1.

Cycle 0 — Theta D[0] Calculation

After $C[x]$, Theta builds a diffusion mask $D[x]$ for each column.

D[0] formula

$C[(x+4) \bmod 5] \text{ XOR } (C[(x+1) \bmod 5])$

D[0] calculation

$D[0] = C[4] \text{ XOR } \text{ROTL1}(C[1])$

Rotate-left by 1 bit

The leftmost bit moves to the far right. This is a rotation, not a normal shift.

$C[1]$ bits before rotation [1] 11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001010 1100101

After ROTL1: 11000010 11010000 11100110 11001010 10011010 11001000 11001010 1100101 1

$\text{ROTL1}(C[1]) = 0xc2d0e6ca9ac8cacb$

Part	64-bit value
$C[4]$	0x0000000000000000
$\text{ROTL1}(C[1])$	0xc2d0e6ca9ac8cacb
D[0] XOR result	0xc2d0e6ca9ac8cacb

How D[0] result is grouped into hex

Each hexadecimal digit represents exactly 4 bits.

c	2	d	0	e	6	c	a	9	a	c	8	c	a	c	b
1100	0010	1101	0000	1110	0110	1100	1010	1001	1010	1100	1000	1100	1010	1100	1011

Binary XOR calculation:

$C[4]$ 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 $\text{ROTL1}(C[1])$ 11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001011
D[0] result 11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001011

Cycle 0 — Theta D[1] Calculation

After C[x], Theta builds a diffusion mask D[x] for each column.

D[1] formula

$$D[1] = C[0] \text{ XOR } \text{ROTL1}(C[2])$$

Rotate-left by 1 bit

The leftmost bit moves to the far right. This is a rotation, not a normal shift.

C[2] bits before rotation [0] 00000000 00001100 11100010 11010010 11100100 11000010 10101000 1101100

After ROTL1: 00000000 00001100 11100010 11010010 11100100 11000010 10101000 1101100 [0]

$$\text{ROTL1}(C[2]) = 0x000ce2d2e4c2a8d8$$

Part	64-bit value
C[0]	0x6a616d6c75646241
ROTL1(C[2])	0x000ce2d2e4c2a8d8
D[1] XOR result	0x6a6d8fbe91a6ca99

How D[1] result is grouped into hex

Each hexadecimal digit represents exactly 4 bits.

6	a	6	d	8	f	b	e	9	1	a	6	c	a	9	9
0110	1010	0110	1101	1000	1111	1011	1110	1001	0001	1010	0110	1100	1010	1001	1001

Binary XOR calculation:

C[0] 01101010 01100001 01101101 01101100 01110101 01100100 01100010 01000001
 ROTL1(C[2]) 00000000 00001100 11100010 11010010 11100100 11000010 10101000 11011000
 D[1] result 01101010 01101101 10001111 10111110 10010001 10100110 11001010 10011001

Cycle 0 — Theta D[2] Calculation

After C[x], Theta builds a diffusion mask D[x] for each column.

D[2] formula

$$D[2] = C[1] \text{ XOR } \text{ROTL1}(C[3])$$

Rotate-left by 1 bit

The leftmost bit moves to the far right. This is a rotation, not a normal shift.

C[3] bits before rotation [0] 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

After ROTL1: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 [0]

$$\text{ROTL1}(C[3]) = 0x0000000000000000$$

Part	64-bit value
C[1]	0xe16873654d646565
ROTL1(C[3])	0x0000000000000000
D[2] XOR result	0xe16873654d646565

How D[2] result is grouped into hex

Each hexadecimal digit represents exactly 4 bits.

e	1	6	8	7	3	6	5	4	d	6	4	6	5	6	5
1110	0001	0110	1000	0111	0011	0110	0101	0100	1101	0110	0100	0110	0101	0110	0101

Binary XOR calculation:

C[1] 11100001 01101000 01110011 01100101 01001101 01100100 01100101 01100101
 ROTL1(C[3]) 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 D[2] result 11100001 01101000 01110011 01100101 01001101 01100100 01100101 01100101

Cycle 0 — Theta D[3] Calculation

After C[x], Theta builds a diffusion mask D[x] for each column.

D[3] formula

$$D[3] = C[2] \text{ XOR } \text{ROTL1}(C[4])$$

Rotate-left by 1 bit

The leftmost bit moves to the far right. This is a rotation, not a normal shift.

C[4] bits before rotation [0] 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

After ROTL1: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 [0]

$$\text{ROTL1}(C[4]) = 0x0000000000000000$$

Part	64-bit value
C[2]	0x000671697261546c
ROTL1(C[4])	0x0000000000000000
D[3] XOR result	0x000671697261546c

How D[3] result is grouped into hex

Each hexadecimal digit represents exactly 4 bits.

0	0	0	6	7	1	6	9	7	2	6	1	5	4	6	c
0000	0000	0000	0110	0111	0001	0110	1001	0111	0010	0110	0001	0101	0100	0110	1100

Binary XOR calculation:

C[2] 00000000 00000110 01110001 01101001 01110010 01100001 01010100 01101100
 ROTL1(C[4]) 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 D[3] result 00000000 00000110 01110001 01101001 01110010 01100001 01010100 01101100

Cycle 0 — Theta D[4] Calculation

After C[x], Theta builds a diffusion mask D[x] for each column.

D[4] formula

$$D[4] = C[3] \text{ XOR } \text{ROTL1}(C[0])$$

Rotate-left by 1 bit

The leftmost bit moves to the far right. This is a rotation, not a normal shift.

C[0] bits before rotation [0] 11010100 11000010 11011010 11011000 11101010 11001000 11000100 1000001

After ROTL1: 11010100 11000010 11011010 11011000 11101010 11001000 11000100 1000001 [0]

$$\text{ROTL1}(C[0]) = 0xd4c2dad8eac8c482$$

Part	64-bit value
C[3]	0x0000000000000000
ROTL1(C[0])	0xd4c2dad8eac8c482
D[4] XOR result	0xd4c2dad8eac8c482

How D[4] result is grouped into hex

Each hexadecimal digit represents exactly 4 bits.

d	4	c	2	d	a	d	8	e	a	c	8	c	4	8	2
1101	0100	1100	0010	1101	1010	1101	1000	1110	1010	1100	1000	1100	0100	1000	0010

Binary XOR calculation:

C[3] 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 ROTL1(C[0]) 11010100 11000010 11011010 11011000 11101010 11001000 11000100 10000010
 D[4] result 11010100 11000010 11011010 11011000 11101010 11001000 11000100 10000010

Cycle 0 — Apply D[x] to Each Column

Final part of Theta: each D[x] is XORed into every lane of column x.

Theta update rule

$$A[x,y] = A[x,y] \text{ XOR } D[x]$$

Column	Old lane example	D[x]	New lane
x=0	0x6a16d6c75646241	0xc2d0e6ca9ac8cacb	0xa8b18ba6efaca88a
x=1	0x616873654d646565	0x6a6d8fbe91a6ca99	0x0b05fcd bcc2affc
x=2	0x000671697261546c	0xe16873654d646565	0xe16e020c3f053109
x=3	0x0000000000000000	0x000671697261546c	0x000671697261546c
x=4	0x0000000000000000	0xd4c2dad8eac8c482	0xd4c2dad8eac8c482

State After Theta

	x=0	x=1	x=2	x=3	x=4
y=0	a8b18ba6 efaca88a	0b05fcd b dcc2affc	e16e020c 3f053109	00067169 7261546c	d4c2dad8 eac8c482
y=1	c2d0e6ca 9ac8cacb	6a6d8fbe 91a6ca99	e1687365 4d646565	00067169 7261546c	d4c2dad8 eac8c482
y=2	c2d0e6ca 9ac8cacb	6a6d8fbe 91a6ca99	e1687365 4d646565	00067169 7261546c	d4c2dad8 eac8c482
y=3	c2d0e6ca 9ac8cacb	ea6d8fbe 91a6ca99	e1687365 4d646565	00067169 7261546c	d4c2dad8 eac8c482
y=4	c2d0e6ca 9ac8cacb	6a6d8fbe 91a6ca99	e1687365 4d646565	00067169 7261546c	d4c2dad8 eac8c482

How we got State After Theta

Use the D value of the same column: $A'[x,y] = A[x,y] \text{ XOR } D[x]$.

Example for first lane $A'[0,0]$:

old A[0,0] 01101010 01100001 01101101 01101100 01110101 01100100 01100010 01000001
D[0] 11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001011
new A'[0,0] 10101000 10110001 10001011 10100110 11101111 10101100 10101000 10001010
Hex result: 0x6a16d6c75646241 XOR 0xc2d0e6ca9ac8cacb = 0xa8b18ba6efaca88a

Binary XOR example for Theta update:

old A[0,0] 01101010 01100001 01101101 01101100 01110101 01100100 01100010 01000001
D[0] 11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001011
new A[0,0] 10101000 10110001 10001011 10100110 11101111 10101100 10101000 10001010

State After Theta — Row y=0

Every cell is calculated with $A'[x,y] = A[x,y] \text{ XOR } D[x]$. Same column means same $D[x]$.

Rule used for every cell

$A'[x,y] = A[x,y] \text{ XOR } D[x] \mid \text{lane size} = 64 \text{ bits} = 8 \text{ bytes}$

D values used by columns

$D[0] = \text{c2d0e6ca9ac8cacb}$ $D[1] = \text{6a6d8fbe91a6ca99}$
 $D[2] = \text{e16873654d646565}$
 $D[3] = \text{000671697261546c}$ $D[4] = \text{d4c2dad8eac8c482}$

Read each block as: old lane bits XOR $D[x]$ bits = new lane bits. The hex result is written under the binary calculation.

Cell $A'[0,0]$

Column $x=0$ uses $D[0] \rightarrow 0x6a616d6c75646241 \text{ XOR } 0xc2d0e6ca9ac8cacb = 0xa8b18ba6efaca88a$

old A 01101010 01100001 01101101 01101100 01110101 01100100 01100010 01000001
 $D[0]$ 11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001011
new A' 10101000 10110001 10001011 10100110 11101111 10101100 10101000 10001010

Result for $A'[0,0] = 0xa8b18ba6efaca88a$

Cell $A'[1,0]$

Column $x=1$ uses $D[1] \rightarrow 0x616873654d646565 \text{ XOR } 0x6a6d8fbe91a6ca99 = 0x0b05fcd bcc2affc$

old A 01100001 01101000 01110011 01100101 01001101 01100100 01100101 01100101
 $D[1]$ 01101010 01101101 10001111 10111110 10010001 10100110 11001010 10011001
new A' 00001011 00000101 11111100 11011011 11011100 11000010 10101111 11111100

Result for $A'[1,0] = 0x0b05fcd bcc2affc$

Cell $A'[2,0]$

Column $x=2$ uses $D[2] \rightarrow 0x000671697261546c \text{ XOR } 0xe16873654d646565 = 0xe16e020c3f053109$

old A 00000000 00000110 01110001 01101001 01110010 01100001 01010100 01101100
 $D[2]$ 11100001 01101000 01110011 01100101 01001101 01100100 01100101 01100101
new A' 11100001 01101110 00000010 00001100 00111111 00000101 00110001 00001001

Result for $A'[2,0] = 0xe16e020c3f053109$

Cell $A'[3,0]$

Column $x=3$ uses $D[3] \rightarrow 0x0000000000000000 \text{ XOR } 0x000671697261546c = 0x000671697261546c$

old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 $D[3]$ 00000000 00000110 01110001 01101001 01110010 01100001 01010100 01101100
new A' 00000000 00000110 01110001 01101001 01110010 01100001 01010100 01101100

Result for $A'[3,0] = 0x000671697261546c$

Cell $A'[4,0]$

Column $x=4$ uses $D[4] \rightarrow 0x0000000000000000 \text{ XOR } 0xd4c2dad8eac8c482 = 0xd4c2dad8eac8c482$

old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 $D[4]$ 11010100 11000010 11011010 11011000 11101010 11001000 11000100 10000010
new A' 11010100 11000010 11011010 11011000 11101010 11001000 11000100 10000010

Result for $A'[4,0] = 0xd4c2dad8eac8c482$

State After Theta — Row y=1

Every cell is calculated with $A'[x,y] = A[x,y] \text{ XOR } D[x]$. Same column means same $D[x]$.

Rule used for every cell

$A'[x,y] = A[x,y] \text{ XOR } D[x] \mid \text{lane size} = 64 \text{ bits} = 8 \text{ bytes}$

D values used by columns

$D[0] = \text{c2d0e6ca9ac8cacb}$ $D[1] = \text{6a6d8fbe91a6ca99}$
 $D[2] = \text{e16873654d646565}$
 $D[3] = \text{000671697261546c}$ $D[4] = \text{d4c2dad8eac8c482}$

Read each block as: old lane bits XOR $D[x]$ bits = new lane bits. The hex result is written under the binary calculation.

Cell $A'[0,1]$

Column $x=0$ uses $D[0] \rightarrow 0x0000000000000000 \text{ XOR } 0xc2d0e6ca9ac8cacb = 0xc2d0e6ca9ac8cacb$

old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 $D[0]$ 11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001011
new A' 11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001011

Result for $A'[0,1] = 0xc2d0e6ca9ac8cacb$

Cell $A'[1,1]$

Column $x=1$ uses $D[1] \rightarrow 0x0000000000000000 \text{ XOR } 0x6a6d8fbe91a6ca99 = 0x6a6d8fbe91a6ca99$

old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 $D[1]$ 01101010 01101101 10001111 10111110 10010001 10100110 11001010 10011001
new A' 01101010 01101101 10001111 10111110 10010001 10100110 11001010 10011001

Result for $A'[1,1] = 0x6a6d8fbe91a6ca99$

Cell $A'[2,1]$

Column $x=2$ uses $D[2] \rightarrow 0x0000000000000000 \text{ XOR } 0xe16873654d646565 = 0xe16873654d646565$

old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 $D[2]$ 11100001 01101000 01110011 01100101 01001101 01100100 01100101 01100101
new A' 11100001 01101000 01110011 01100101 01001101 01100100 01100101 01100101

Result for $A'[2,1] = 0xe16873654d646565$

Cell $A'[3,1]$

Column $x=3$ uses $D[3] \rightarrow 0x0000000000000000 \text{ XOR } 0x000671697261546c = 0x000671697261546c$

old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 $D[3]$ 00000000 00000110 01110001 01101001 01110010 01100001 01010100 01101100
new A' 00000000 00000110 01110001 01101001 01110010 01100001 01010100 01101100

Result for $A'[3,1] = 0x000671697261546c$

Cell $A'[4,1]$

Column $x=4$ uses $D[4] \rightarrow 0x0000000000000000 \text{ XOR } 0xd4c2dad8eac8c482 = 0xd4c2dad8eac8c482$

old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 $D[4]$ 11010100 11000010 11011010 11011000 11101010 11001000 11000100 10000010
new A' 11010100 11000010 11011010 11011000 11101010 11001000 11000100 10000010

Result for $A'[4,1] = 0xd4c2dad8eac8c482$

State After Theta — Row y=2

Every cell is calculated with $A'[x,y] = A[x,y] \text{ XOR } D[x]$. Same column means same $D[x]$.

Rule used for every cell

$A'[x,y] = A[x,y] \text{ XOR } D[x] \mid \text{lane size} = 64 \text{ bits} = 8 \text{ bytes}$

D values used by columns

$D[0] = \text{c2d0e6ca9ac8cacb}$ $D[1] = \text{6a6d8fbe91a6ca99}$
 $D[2] = \text{e16873654d646565}$
 $D[3] = \text{000671697261546c}$ $D[4] = \text{d4c2dad8eac8c482}$

Read each block as: old lane bits XOR $D[x]$ bits = new lane bits. The hex result is written under the binary calculation.

Cell $A'[0,2]$

Column $x=0$ uses $D[0] \rightarrow 0x0000000000000000 \text{ XOR } 0xc2d0e6ca9ac8cacb = 0xc2d0e6ca9ac8cacb$

```
old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
D[0] 11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001011
new A' 11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001011
```

Result for $A'[0,2] = 0xc2d0e6ca9ac8cacb$

Cell $A'[1,2]$

Column $x=1$ uses $D[1] \rightarrow 0x0000000000000000 \text{ XOR } 0x6a6d8fbe91a6ca99 = 0x6a6d8fbe91a6ca99$

```
old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
D[1] 01101010 01101101 10001111 10111110 10010001 10100110 11001010 10011001
new A' 01101010 01101101 10001111 10111110 10010001 10100110 11001010 10011001
```

Result for $A'[1,2] = 0x6a6d8fbe91a6ca99$

Cell $A'[2,2]$

Column $x=2$ uses $D[2] \rightarrow 0x0000000000000000 \text{ XOR } 0xe16873654d646565 = 0xe16873654d646565$

```
old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
D[2] 11100001 01101000 01110011 01100101 01001101 01100100 01100101 01100101
new A' 11100001 01101000 01110011 01100101 01001101 01100100 01100101 01100101
```

Result for $A'[2,2] = 0xe16873654d646565$

Cell $A'[3,2]$

Column $x=3$ uses $D[3] \rightarrow 0x0000000000000000 \text{ XOR } 0x000671697261546c = 0x000671697261546c$

```
old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
D[3] 00000000 00000110 01110001 01101001 01110010 01100001 01010100 01101100
new A' 00000000 00000110 01110001 01101001 01110010 01100001 01010100 01101100
```

Result for $A'[3,2] = 0x000671697261546c$

Cell $A'[4,2]$

Column $x=4$ uses $D[4] \rightarrow 0x0000000000000000 \text{ XOR } 0xd4c2dad8eac8c482 = 0xd4c2dad8eac8c482$

```
old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
D[4] 11010100 11000010 11011010 11011000 11101010 11001000 11000100 10000010
new A' 11010100 11000010 11011010 11011000 11101010 11001000 11000100 10000010
```

Result for $A'[4,2] = 0xd4c2dad8eac8c482$

State After Theta — Row y=3

Every cell is calculated with $A'[x,y] = A[x,y] \text{ XOR } D[x]$. Same column means same $D[x]$.

Rule used for every cell

$A'[x,y] = A[x,y] \text{ XOR } D[x] \mid \text{lane size} = 64 \text{ bits} = 8 \text{ bytes}$

D values used by columns

$D[0]=c2d0e6ca9ac8cacb$ $D[1]=6a6d8fbe91a6ca99$
 $D[2]=e16873654d646565$
 $D[3]=000671697261546c$ $D[4]=d4c2dad8eac8c482$

Read each block as: old lane bits XOR $D[x]$ bits = new lane bits. The hex result is written under the binary calculation.

Cell $A'[0,3]$

Column $x=0$ uses $D[0] \rightarrow 0x0000000000000000 \text{ XOR } 0xc2d0e6ca9ac8cacb = 0xc2d0e6ca9ac8cacb$

```
old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
D[0] 11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001011
new A' 11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001011
```

Result for $A'[0,3] = 0xc2d0e6ca9ac8cacb$

Cell $A'[1,3]$

Column $x=1$ uses $D[1] \rightarrow 0x8000000000000000 \text{ XOR } 0x6a6d8fbe91a6ca99 = 0xea6d8fbe91a6ca99$

```
old A 10000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
D[1] 01101010 01101101 10001111 10111110 10010001 10100110 11001010 10011001
new A' 11101010 01101101 10001111 10111110 10010001 10100110 11001010 10011001
```

Result for $A'[1,3] = 0xea6d8fbe91a6ca99$

Cell $A'[2,3]$

Column $x=2$ uses $D[2] \rightarrow 0x0000000000000000 \text{ XOR } 0xe16873654d646565 = 0xe16873654d646565$

```
old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
D[2] 11100001 01101000 01110011 01100101 01001101 01100100 01100101 01100101
new A' 11100001 01101000 01110011 01100101 01001101 01100100 01100101 01100101
```

Result for $A'[2,3] = 0xe16873654d646565$

Cell $A'[3,3]$

Column $x=3$ uses $D[3] \rightarrow 0x0000000000000000 \text{ XOR } 0x000671697261546c = 0x000671697261546c$

```
old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
D[3] 00000000 00000110 01110001 01101001 01110010 01100001 01010100 01101100
new A' 00000000 00000110 01110001 01101001 01110010 01100001 01010100 01101100
```

Result for $A'[3,3] = 0x000671697261546c$

Cell $A'[4,3]$

Column $x=4$ uses $D[4] \rightarrow 0x0000000000000000 \text{ XOR } 0xd4c2dad8eac8c482 = 0xd4c2dad8eac8c482$

```
old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
D[4] 11010100 11000010 11011010 11011000 11101010 11001000 11000100 10000010
new A' 11010100 11000010 11011010 11011000 11101010 11001000 11000100 10000010
```

Result for $A'[4,3] = 0xd4c2dad8eac8c482$

State After Theta — Row y=4

Every cell is calculated with $A'[x,y] = A[x,y] \text{ XOR } D[x]$. Same column means same $D[x]$.

Rule used for every cell

$A'[x,y] = A[x,y] \text{ XOR } D[x] \mid \text{lane size} = 64 \text{ bits} = 8 \text{ bytes}$

D values used by columns

$D[0]=\text{c2d0e6ca9ac8cacb}$ $D[1]=\text{6a6d8fbe91a6ca99}$
 $D[2]=\text{e16873654d646565}$
 $D[3]=\text{000671697261546c}$ $D[4]=\text{d4c2dad8eac8c482}$

Read each block as: old lane bits XOR $D[x]$ bits = new lane bits. The hex result is written under the binary calculation.

Cell $A'[0,4]$

Column $x=0$ uses $D[0] \rightarrow 0x0000000000000000 \text{ XOR } 0xc2d0e6ca9ac8cacb = 0xc2d0e6ca9ac8cacb$

old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 $D[0]$ 11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001011
new A' 11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001011

Result for $A'[0,4] = 0xc2d0e6ca9ac8cacb$

Cell $A'[1,4]$

Column $x=1$ uses $D[1] \rightarrow 0x0000000000000000 \text{ XOR } 0x6a6d8fbe91a6ca99 = 0x6a6d8fbe91a6ca99$

old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 $D[1]$ 01101010 01101101 10001111 10111110 10010001 10100110 11001010 10011001
new A' 01101010 01101101 10001111 10111110 10010001 10100110 11001010 10011001

Result for $A'[1,4] = 0x6a6d8fbe91a6ca99$

Cell $A'[2,4]$

Column $x=2$ uses $D[2] \rightarrow 0x0000000000000000 \text{ XOR } 0xe16873654d646565 = 0xe16873654d646565$

old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 $D[2]$ 11100001 01101000 01110011 01100101 01001101 01100100 01100101 01100101
new A' 11100001 01101000 01110011 01100101 01001101 01100100 01100101 01100101

Result for $A'[2,4] = 0xe16873654d646565$

Cell $A'[3,4]$

Column $x=3$ uses $D[3] \rightarrow 0x0000000000000000 \text{ XOR } 0x000671697261546c = 0x000671697261546c$

old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 $D[3]$ 00000000 00000110 01110001 01101001 01110010 01100001 01010100 01101100
new A' 00000000 00000110 01110001 01101001 01110010 01100001 01010100 01101100

Result for $A'[3,4] = 0x000671697261546c$

Cell $A'[4,4]$

Column $x=4$ uses $D[4] \rightarrow 0x0000000000000000 \text{ XOR } 0xd4c2dad8eac8c482 = 0xd4c2dad8eac8c482$

old A 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
 $D[4]$ 11010100 11000010 11011010 11011000 11101010 11001000 11000100 10000010
new A' 11010100 11000010 11011010 11011000 11101010 11001000 11000100 10000010

Result for $A'[4,4] = 0xd4c2dad8eac8c482$

Cycle 0 — Rho Step: Bit Rotations

Rho rotates every 64-bit lane by its fixed offset. The unit is bits, not bytes.

Rotation formula

$$\text{Rotated}[x,y] = \text{ROTL64}(A[x,y], \text{ROT}[x,y])$$

y\x	x=0	x=1	x=2	x=3	x=4
y=0	0	1	62	28	27
y=1	36	44	6	55	20
y=2	3	10	43	25	39
y=3	41	45	15	21	8
y=4	18	2	61	56	14

Example to remember: lane (1,0) uses $\text{ROT}[0][1] = 1$ bit, so it moves the first bit to the end.

Next pages show actual bit rotation examples, then Pi moves each rotated lane to a new position.

Cycle 0 — Rho Example: Lane (0,0)

This page shows the rotation at bit level and then groups the result back into hex.

Old lane value

$A[0,0] = 0xa8b18ba6efaca88a$

Rotation amount

ROTL64 by 0 bits

Rotation by 0 means the lane does not change.

Rotated result = 0xa8b18ba6efaca88a

Convert rotated bits back to hexadecimal

Each hexadecimal digit represents exactly 4 bits.

a	8	b	1	8	b	a	6	e	f	a	c	a	8	8	a
1010	1000	1011	0001	1000	1011	1010	0110	1110	1111	1010	1100	1010	1000	1000	1010

Pi destination

New position = $(y, (2x + 3y) \bmod 5) = (0, (2 \cdot 0 + 3 \cdot 0) \bmod 5) = (0, 0)$

Cycle 0 — Rho Example: Lane (1,0)

This page shows the rotation at bit level and then groups the result back into hex.

Old lane value

$A[1,0] = 0x0b05fcdbdcc2affc$

Rotation amount

ROTL64 by 1 bit

Before rotation: shifted-out/wrapped part is marked in brackets [0] 00010110 00001011 11111001 10110111 10111001 10000101 01011111 1111100

After rotation: remaining bits move left, bracketed part wraps to the right 00010110 00001011 11111001 10110111 10111001 10000101 01011111 1111100 [0]

Rotated result = 0x160bf9b7b9855ff8

Convert rotated bits back to hexadecimal

Each hexadecimal digit represents exactly 4 bits.

1	6	0	b	f	9	b	7	b	9	8	5	5	f	f	8
0001	0110	0000	1011	1111	1001	1011	0111	1011	1001	1000	0101	0101	1111	1111	1000

Pi destination

New position = $(y, (2x + 3y) \bmod 5) = (0, (2 \cdot 1 + 3 \cdot 0) \bmod 5) = (0, 2)$

Cycle 0 — Rho Example: Lane (2,0)

This page shows the rotation at bit level and then groups the result back into hex.

Old lane value

$A[2,0] = 0xe16e020c3f053109$

Rotation amount

ROTL64 by 62 bits

Before rotation: shifted-out/wrapped part is marked in brackets [11100001 01101110 00000010 00001100 00111111 00000101 00110001 000010] 01

After rotation: remaining bits move left, bracketed part wraps to the right 01 [11100001 01101110 00000010 00001100 00111111 00000101 00110001 000010]

Rotated result = 0x785b80830fc14c42

Convert rotated bits back to hexadecimal

Each hexadecimal digit represents exactly 4 bits.

7	8	5	b	8	0	8	3	0	f	c	1	4	c	4	2
0111	1000	0101	1011	1000	0000	1000	0011	0000	1111	1100	0001	0100	1100	0100	0010

Pi destination

New position = $(y, (2x + 3y) \bmod 5) = (0, (2 \cdot 2 + 3 \cdot 0) \bmod 5) = (0, 4)$

Cycle 0 — Pi Lane Movement Table (1/2)

Pi rearranges the rotated lanes using the position formula: $\text{New}(x,y) = (\text{old } y, (2 \cdot \text{old } x + 3 \cdot \text{old } y) \bmod 5)$.

Pi formula

$\text{new position} = (y, (2x + 3y) \bmod 5)$

Old lane	Rot (bits)	Rotated 64-bit value	New lane
(0, 0)	0	0xa8b18ba6efaca88a	(0, 0)
(1, 0)	1	0x160bf9b7b9855ff8	(0, 2)
(2, 0)	62	0x785b80830fc14c42	(0, 4)
(3, 0)	28	0x97261546c0006716	(0, 1)
(4, 0)	27	0xc756462416a616d6	(0, 3)
(0, 1)	36	0xac8cacbc2d0e6ca9	(1, 3)
(1, 1)	44	0x6ca996a6d8f9e91a	(1, 0)
(2, 1)	6	0x5a1cd95359195978	(1, 2)
(3, 1)	55	0x36000338b4b930aa	(1, 4)
(4, 1)	20	0xad8eac8c482d4c2d	(1, 1)
(0, 2)	3	0x16873654d646565e	(2, 1)
(1, 2)	10	0xb63efa469b2a65a9	(2, 3)
(2, 2)	43	0x232b2f0b439b2a6b	(2, 0)

Notice that Rho changes bit positions inside each lane, while Pi changes the lane location in the 5×5 state.

Cycle 0 — Pi Lane Movement Table (2/2)

Pi rearranges the rotated lanes using the position formula: $\text{New}(x,y) = (\text{old } y, (2 \cdot \text{old } x + 3 \cdot \text{old } y) \bmod 5)$.

Pi formula

new position = (y, (2x + 3y) mod 5)

Old lane	Rot (bits)	Rotated 64-bit value	New lane
(3, 2)	25	0xd2e4c2a8d8000ce2	(2, 2)
(4, 2)	39	0x6462416a616d6c75	(2, 4)
(0, 3)	41	0x91959785a1cd9535	(3, 4)
(1, 3)	45	0xd9533d4db1f7d234	(3, 1)
(2, 3)	15	0x39b2a6b232b2f0b4	(3, 3)
(3, 3)	21	0x2d2e4c2a8d8000ce	(3, 0)
(4, 3)	8	0xc2dad8eac8c482d4	(3, 2)
(0, 4)	18	0x9b2a6b232b2f0b43	(4, 2)
(1, 4)	2	0xa9b63efa469b2a65	(4, 4)
(2, 4)	61	0xbc2d0e6ca9ac8cac	(4, 1)
(3, 4)	56	0x6c00067169726154	(4, 3)
(4, 4)	14	0xb6b63ab23120b530	(4, 0)

Notice that Rho changes bit positions inside each lane, while Pi changes the lane location in the 5×5 state.

Cycle 0 — State After Rho-Pi

After Rho rotates each lane and Pi moves it, the state is rearranged as shown below.

State After Rho-Pi

	x=0	x=1	x=2	x=3	x=4
y=0	a8b18ba6 efaca88a	6ca996a6 d8fbe91a	232b2f0b 439b2a6b	2d2e4c2a 8d8000ce	b6b63ab2 3120b530
y=1	97261546 c0006716	ad8eac8c 482d4c2d	16873654 d646565e	d9533d4d b1f7d234	bc2d0e6c a9ac8cac
y=2	160bf9b7 b9855ff8	5a1cd953 59195978	d2e4c2a8 d8000ce2	c2dad8ea c8c482d4	9b2a6b23 2b2f0b43
y=3	c7564624 16a616d6	ac8cacbc 2d0e6ca9	b63efa46 9b2a65a9	39b2a6b2 32b2f0b4	6c000671 69726154
y=4	785b8083 0fc14c42	36000338 b4b930aa	6462416a 616d6c75	91959785 a1cd9535	a9b63efa 469b2a65

Highlighted first column shows how old row y=0 values moved into x=0 positions after Pi.

Next: Chi processes one row at a time and introduces the non-linear part of SHA3.

Cycle 0 — Chi Step: Non-linear Row Mixing

Chi is the only non-linear step in a Keccak round. It operates independently on each row.

Chi formula

```
new[x] = old[x] XOR ((NOT old[(x+1) mod 5]) AND old[(x+2) mod 5])
```

How to read the formula

1

Take current lane old[x].

2

Look at the next two lanes in the same row: old[x+1] and old[x+2].

3

Invert old[x+1] bit-by-bit using NOT.

4

AND that result with old[x+2].

5

XOR with old[x] to produce new[x].

Important: Chi wraps around at the end of the row. For x=4, the next lanes are x=0 and x=1.

Input to Chi: State After Rho-Pi

	x=0	x=1	x=2	x=3	x=4
y=0	a8b18ba6 efaca88a	6ca996a6 d8fbe91a	232b2f0b 439b2a6b	2d2e4c2a 8d8000ce	b6b63ab2 3120b530
y=1	97261546 c0006716	ad8eac8c 482d4c2d	16873654 d646565e	d9533d4d b1f7d234	bc2d0e6c a9ac8cac
y=2	160bf9b7 b9855ff8	5a1cd953 59195978	d2e4c2a8 d8000ce2	c2dad8ea c8c482d4	9b2a6b23 2b2f0b43
y=3	c7564624 16a616d6	ac8cacbc 2d0e6ca9	b63efa46 9b2a65a9	39b2a6b2 32b2f0b4	6c000671 69726154
y=4	785b8083 0fc14c42	36000338 b4b930aa	6462416a 616d6c75	91959785 a1cd9535	a9b63efa 469b2a65

Cycle 0 — Chi Row 0 Detailed Calculation

Row y=0: calculate x=0..4 using NOT, AND, and XOR. All values are 64-bit lanes.

x	old[x]	NOT old[x+1]	AND with old[x+2]	new[x]
0	0xa8b18ba6efaca88a	0x93566959270416e5	0x0302290903000261	0xabb3a2afecacaaeb
1	0x6ca996a6d8fbe91a	0xdcd4d0f4bc64d594	0x0c0440208c000084	0x60add68654fbe99e
2	0x232b2f0b439b2a6b	0xd2d1b3d5727fff31	0x929032903020b530	0xb1bb1d9b73bb9f5b
3	0x2d2e4c2a8d8000ce	0x4949c54dcedf4acf	0x08018104ce8c088a	0x252fcd2e430c0844
4	0xb6b63ab23120b530	0x574e745910535775	0x4408140010534110	0xf2be2eb22173f420

Binary NOT / AND / XOR example for row 0, x=0

```
old[0] = 10101000 10110001 10001011 10100110 11101111 10101100 10101000 10001010 old[1] =
01101100 10101001 10010110 10100110 11011000 11111011 11101001 00011010 NOT old[1] = 10010011
01010110 01101001 01011001 00100111 00000100 00010110 11100101 old[2] = 00100011 00101011
00101111 00001011 01000011 10011011 00101010 01101011 (NOT old[1]) AND old[2] = 00000011
00000010 00101001 00001001 00000011 00000000 00000010 01100001 old[0] XOR mask = 10101011
10110011 10100010 10101111 11101100 10101100 10101010 11101011
```

Row 0 output: 0xabb3a2afecacaaeb | 0x60add68654fbe99e | 0xb1bb1d9b73bb9f5b | 0x252fcd2e430c0844 | 0xf2be2eb22173f420

Chi meaning: a lane is changed only by information from its two neighbors in the same row.

Cycle 0 — Chi Row 1 Detailed Calculation

Row y=1: calculate x=0..4 using NOT, AND, and XOR. All values are 64-bit lanes.

x	old[x]	NOT old[x+1]	AND with old[x+2]	new[x]
0	0x97261546c0006716	0x52715373b7d2b3d2	0x1201125096421252	0x8527071656427544
1	0xad8eac8c482d4c2d	0xe978c9ab29b9a9a1	0xc950090921b18020	0x64dea585699ccc0d
2	0x16873654d646565e	0x26acc2b24e082dcb	0x242c022008080c88	0x32ab3474de4e5ad6
3	0xd9533d4db1f7d234	0x43d2f19356537353	0x0302110240006312	0xda512c4ff1f7b126
4	0xbc2d0e6ca9ac8cac	0x68d9eab93fff98e9	0x2888a888082d0829	0x94a5a6e4a1818485

Binary NOT / AND / XOR example for row 1, x=0

```
old[0] = 10010111 00100110 00010101 01000110 11000000 00000000 01100111 00010110 old[1] =
10101101 10001110 10101100 10001100 01001000 00101101 01001100 00101101 NOT old[1] = 01010010
01110001 01010011 01110011 10110111 11010010 10110011 11010010 old[2] = 00010110 10000111
00110110 01010100 11010110 01000110 01010110 01011110 (NOT old[1]) AND old[2] = 00010010
00000001 00010010 01010000 10010110 01000010 00010010 01010010 old[0] XOR mask = 10000101
00100111 00000111 00010110 01010110 01000010 01110101 01000100
```

Row 1 output: 0x8527071656427544 | 0x64dea585699ccc0d | 0x32ab3474de4e5ad6 | 0xda512c4ff1f7b126 | 0x94a5a6e4a1818485

Chi meaning: a lane is changed only by information from its two neighbors in the same row.

Cycle 0 — Chi Row 2 Detailed Calculation

Row y=2: calculate x=0..4 using NOT, AND, and XOR. All values are 64-bit lanes.

x	old[x]	NOT old[x+1]	AND with old[x+2]	new[x]
0	0x160bf9b7b9855ff8	0xa5e326aca6e6a687	0x80e002a880000482	0x96ebfb1f39855b7a
1	0x5a1cd95359195978	0x2d1b3d5727fff31d	0x001a184200c48214	0x5a06c11159dddb6c
2	0xd2e4c2a8d8000ce2	0x3d252715373b7d2b	0x19202301232b0903	0xcbc4e1a9fb2b05e1
3	0xc2dad8eac8c482d4	0x64d594dcd4d0f4bc	0x04019094908054b8	0xc6db487e5844d66c
4	0x9b2a6b232b2f0b43	0xe9f40648467aa007	0x4814004040180000	0xd33e6b636b370b43

Binary NOT / AND / XOR example for row 2, x=0

```
old[0] = 00010110 00001011 11111001 10110111 10111001 10000101 01011111 11111000 old[1] =
01011010 00011100 11011001 01010011 01011001 00011001 01011001 01111000 NOT old[1] = 10100101
11100011 00100110 10101100 10100110 11100110 10100110 10000111 old[2] = 11010010 11100100
11000010 10101000 11011000 00000000 00001100 11100010 (NOT old[1]) AND old[2] = 10000000
11100000 00000010 10101000 10000000 00000000 00000100 10000010 old[0] XOR mask = 10010110
11101011 11111011 00011111 00111001 10000101 01011011 01111010
```

Row 2 output: 0x96ebfb1f39855b7a | 0x5a06c11159dddb6c | 0xcbc4e1a9fb2b05e1 | 0xc6db487e5844d66c | 0xd33e6b636b370b43

Chi meaning: a lane is changed only by information from its two neighbors in the same row.

Cycle 0 — Chi Row 3 Detailed Calculation

Row y=3: calculate x=0..4 using NOT, AND, and XOR. All values are 64-bit lanes.

x	old[x]	NOT old[x+1]	AND with old[x+2]	new[x]
0	0xc756462416a616d6	0x53735343d2f19356	0x1232524292200100	0xd5641466848617d6
1	0xac8cacbc2d0e6ca9	0x49c105b964d59a56	0x098004b020909014	0xa50ca80c0d9efcbd
2	0xb63efa469b2a65a9	0xc64d594dcd4d0f4b	0x4400004149400140	0xf23efa07d26a64e9
3	0x39b2a6b232b2f0b4	0x93fff98e968d9eab	0x8356400416841682	0xbae4e6b62436e636
4	0x6c00067169726154	0x38a9b9dbe959e929	0x2888a89829086829	0x4488aee9407a097d

Binary NOT / AND / XOR example for row 3, x=0

```
old[0] = 11000111 01010110 01000110 00100100 00010110 10100110 00010110 11010110 old[1] =
10101100 10001100 10101100 10111100 00101101 00001110 01101100 10101001 NOT old[1] = 01010011
01110011 01010011 01000011 11010010 11110001 10010011 01010110 old[2] = 10110110 00111110
11111010 01000110 10011011 00101010 01100101 10101001 (NOT old[1]) AND old[2] = 00010010
00110010 01010010 01000010 10010010 00100000 00000001 00000000 old[0] XOR mask = 11010101
01100100 00010100 01100110 10000100 10000110 00010111 11010110
```

Row 3 output: 0xd5641466848617d6 | 0xa50ca80c0d9efcbd | 0xf23efa07d26a64e9 | 0xbae4e6b62436e636 | 0x4488aee9407a097d

Chi meaning: a lane is changed only by information from its two neighbors in the same row.

Cycle 0 — Chi Row 4 Detailed Calculation

Row y=4: calculate x=0..4 using NOT, AND, and XOR. All values are 64-bit lanes.

x	old[x]	NOT old[x+1]	AND with old[x+2]	new[x]
0	0x785b80830fc14c42	0xc9ffffcc74b46cf55	0x4062404241444c55	0x3839c0c14e850017
1	0x36000338b4b930aa	0x9b9dbe959e92938a	0x9195968580809100	0xa79595bd3439a1aa
2	0x6462416a616d6c75	0x6e6a687a5e326aca	0x2822287a46122a40	0x4c406910277f4635
3	0x91959785a1cd9535	0x5649c105b964d59a	0x5049800109404402	0xc1dc1784a88dd137
4	0xa9b63efa469b2a65	0x87a47f7cf03eb3bd	0x06000338b03830a8	0xafb63dc2f6a31acd

Binary NOT / AND / XOR example for row 4, x=0

```
old[0] = 01111000 01011011 10000000 10000011 00001111 11000001 01001100 01000010 old[1] =
00110110 00000000 00000011 00111000 10110100 10111001 00110000 10101010 NOT old[1] = 11001001
11111111 11111100 11000111 01001011 01000110 11001111 01010101 old[2] = 01100100 01100010
01000001 01101010 01100001 01101101 01101100 01110101 (NOT old[1]) AND old[2] = 01000000
01100010 01000000 01000010 01000001 01000100 01001100 01010101 old[0] XOR mask = 00111000
00111001 11000000 11000001 01001110 10000101 00000000 00010111
```

Row 4 output: 0x3839c0c14e850017 | 0xa79595bd3439a1aa | 0x4c406910277f4635 | 0xc1dc1784a88dd137 | 0xafb63dc2f6a31acd

Chi meaning: a lane is changed only by information from its two neighbors in the same row.

Cycle 0 — State After Chi

All five rows have now been non-linearly mixed.

State After Chi

	x=0	x=1	x=2	x=3	x=4
y=0	abb3a2af ecacaaeb	60add686 54fbe99e	b1bb1d9b 73bb9f5b	252fcd2e 430c0844	f2be2eb2 2173f420
y=1	85270716 56427544	64dea585 699ccc0d	32ab3474 de4e5ad6	da512c4f f1f7b126	94a5a6e4 a1818485
y=2	96ebfb1f 39855b7a	5a06c111 59dddb6c	cbc4e1a9 fb2b05e1	c6db487e 5844d66c	d33e6b63 6b370b43
y=3	d5641466 848617d6	a50ca80c 0d9efcbd	f23efa07 d26a64e9	bae4e6b6 2436e636	4488aee9 407a097d
y=4	3839c0c1 4e850017	a79595bd 3439a1aa	4c406910 277f4635	c1dc1784 a88dd137	afb63dc2 f6a31acd

Next: Iota adds the round constant RC[0] only to lane A[0,0].

Cycle 0 — Iota Step and Final State

Iota breaks round symmetry by XORing the round constant into A[0,0].

Iota formula

$$A[0,0] = A[0,0] \text{ XOR } RC[0]$$

Part	64-bit value
A[0,0] before Iota	0xabb3a2afecacaaeb
RC[0]	0x0000000000000001
XOR result	0xabb3a2afecacaaea

Binary XOR calculation

A[0,0] before = 10101011 10110011 10100010 10101111 11101100 10101100 10101010 11101011 RC[0] = 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000001 XOR result = 10101011 10110011 10100010 10101111 11101100 10101100 10101010 11101010

Only the last bit changed: ...1011 XOR ...0001 = ...1010. Therefore 0x...aaeb becomes 0x...aaea.

Final State After Cycle 0

	x=0	x=1	x=2	x=3	x=4
y=0	abb3a2af ecacaaea	60add686 54f9e99e	b1bb1d9b 73bb9f5b	252fcd2e 430c0844	f2be2eb2 2173f420
y=1	85270716 56427544	64dea585 699ccc0d	32ab3474 de4e5ad6	da512c4f f1f7b126	94a5a6e4 a1818485
y=2	96ebfb1f 39855b7a	5a06c111 59dddb6c	cbc4e1a9 fb2b05e1	c6db487e 5844d66c	d33e6b63 6b370b43
y=3	d5641466 848617d6	a50ca80c 0d9efcbd	f23efa07 d26a64e9	bae4e6b6 2436e636	4488aee9 407a097d
y=4	3839c0c1 4e850017	a79595bd 3439a1aa	4c406910 277f4635	c1dc1784 a88dd137	afb63dc2 f6a31acd

Compact Bitwise Notes

This replaces the too-long 42-page bitwise appendix with grouped examples.

Why not every lane gets its own page?

Showing all 25 Chi lanes + every Theta XOR separately creates a huge file. This compact version shows the full method with representative bit-level calculations, while keeping the complete state tables from the main pages.

XOR rule

$$0 \text{ XOR } 0 = 0$$

$$0 \text{ XOR } 1 = 1$$

$$1 \text{ XOR } 0 = 1$$

$$1 \text{ XOR } 1 = 0$$

Use this rule vertically across 64 bit positions. Hex is only the final compact representation.

Theta C[0] Bitwise XOR

C[0] is XOR across the five lanes of column x=0.

A[0,0]

01101010 01100001 01101101 01101100 01110101 01100100 01100010 01000001

A[0,1]

00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

A[0,2]

00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

A[0,3]

00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

A[0,4]

00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

C[0] result

01101010 01100001 01101101 01101100 01110101 01100100 01100010 01000001

Since the other four lanes are zero, the result remains A[0,0].

Theta D[0] Bitwise: Rotate then XOR

$D[0] = C[4] \text{ XOR } \text{ROTL1}(C[1]).$

ROTL1(C[1])

Before: [1] 11000010 11010000 11100110 11001010 10011010 11001000 11001010 1100101

After : 11000010 11010000 11100110 11001010 10011010 11001000 11001010 1100101 [1]

Now XOR

C[4]

00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

ROTL1(C[1])

11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001011

D[0] result

11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001011

Hex result: 0xc2d0e6ca9ac8cacb

Apply D[0] Bitwise to A[0,0]

Example of the Theta update rule: $A[x,y] = A[x,y] \text{ XOR } D[x]$.

Old A[0,0]

01101010 01100001 01101101 01101100 01110101 01100100 01100010 01000001

D[0]

11000010 11010000 11100110 11001010 10011010 11001000 11001010 11001011

New A[0,0]

10101000 10110001 10001011 10100110 11101111 10101100 10101000 10001010

The same pattern is applied to every lane in column $x=0$, and similarly for $D[1]$ through $D[4]$.

Chi Bitwise Example: Row 0, x=0

$\text{new}[0] = \text{old}[0] \text{ XOR } ((\text{NOT } \text{old}[1]) \text{ AND } \text{old}[2]).$

old[0]

10101000 10110001 10001011 10100110 11101111 10101100 10101000 10001010

old[1]

01101100 10101001 10010110 10100110 11011000 11111011 11101001 00011010

NOT old[1]

10010011 01010110 01101001 01011001 00100111 00000100 00010110 11100101

old[2]

00100011 00101011 00101111 00001011 01000011 10011011 00101010 01101011

mask

00000011 00000010 00101001 00001001 00000011 00000000 00000010 01100001

new[0]

10101011 10110011 10100010 10101111 11101100 10101100 10101010 11101011

This single example explains the exact bitwise process used for every Chi lane.

Iota Bitwise XOR with RC[0]

Iota changes only A[0,0] by XORing it with RC[0].

A[0,0] before

10101011 10110011 10100010 10101111 11101100 10101100 10101010 11101011

RC[0]

00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000001

A[0,0] after

10101011 10110011 10100010 10101111 11101100 10101100 10101010 11101010

Last byte: eb XOR 01 = ea → 11101011 XOR 00000001 = 11101010.